



DevOps

Cloud Adoption and Cloud Native
Architecture



Bibliography

Laszewski, Tom; Arora, Kamal; Farr, Erik; Zonooz, Piyum. Cloud Native Architectures: Design high-availability and cost-effective applications for the cloud (p. 8). Packt Publishing



Cloud Adoption Drivers

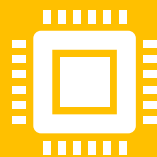
Moving fast and constraining costs



Starting out with the smallest investment possible



Aiming for exponential growth



Cloud computing is being able to provision resources on demand and remove those resources when they're no longer required

Agility

Pre-cloud common issue: long lead times in deploying hardware to their data centers, causing all kinds of issues, including the expansion of project timelines

Cloud offers virtually limitless capacity and ability to use the capacity within minutes

Allows the organization to develop a market strategy based on their core competencies

Enables failing fast (*"It's not an experiment if you know it's going to work"* – Jeff Bezos)

Enables resources to be created and destroyed exactly when needed, or when no longer needed

Cost



High cost of experimentation will impact company



Cloud computing provides the cost benefits of being agile, coupled with minimal upfront costs



Full life cycle of a new business idea can be weeks/months (very few resources at the beginning and ability to scale up and down based on success)



Fine-grained billing mechanisms (seconds with aggregated monthly bills)



Monthly bills recognized as operational costs (no complex capital approvals needed)



Inherit from cloud provider

Being secure and maintaining
proper governance

Security



Cloud is undeniably incredibly secure when properly configured



Cloud vendors employ hundreds or even thousands of people whose sole job is to think about how to more securely manage their environment



Proactive monitoring and automated resolution of misconfigured controls using AI and ML



Vast majority of companies cannot deploy the resources required to continuously innovate their security like cloud vendors can



Is focus on security really the best use of an organizations resources, or should they be focused on business results

Governance



Cloud vendors have lots of services designed to meet just about every security requirement



It is up to the individual systems to implement those services according to their design specifications



The cloud offers services that interact with the other services to form an overlay of monitoring, auditing, configuration controls, and other governance activities



Using the cloud will enable the workloads that are deployed by the company to all have the same governance model



Cloud Migrations Patterns

Cloud Migrations Patterns

Rehost

Replatform

Repurchase

Refactor

Retire

Retain

Rehost (Lift-And-Shift)



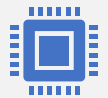
Quickest path to cloud



Moving a workload exactly as is from its existing location (usually on-premises) to the cloud



Tools that enable this strategy including databases

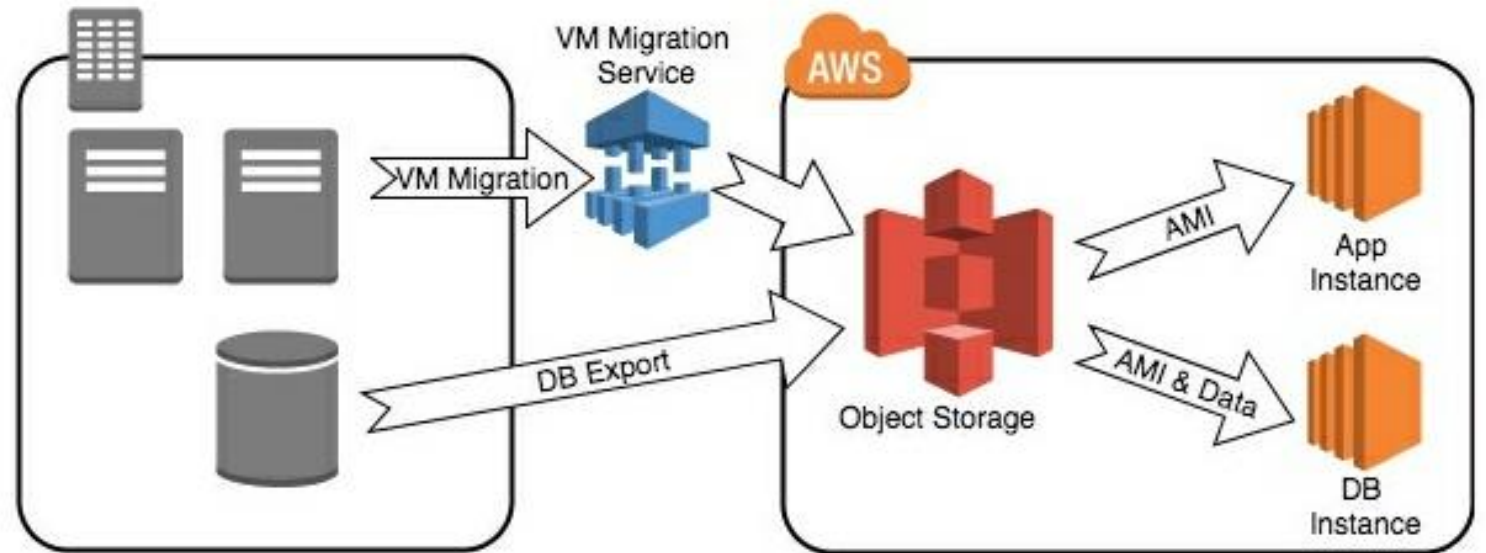


The drawback of this type of migration is that typically, the speed of migration means that the true benefits of the cloud are not being realized



Will accelerate the ability to replatform or refactor applications

Rehost (Lift-And-Shift)



Replatform

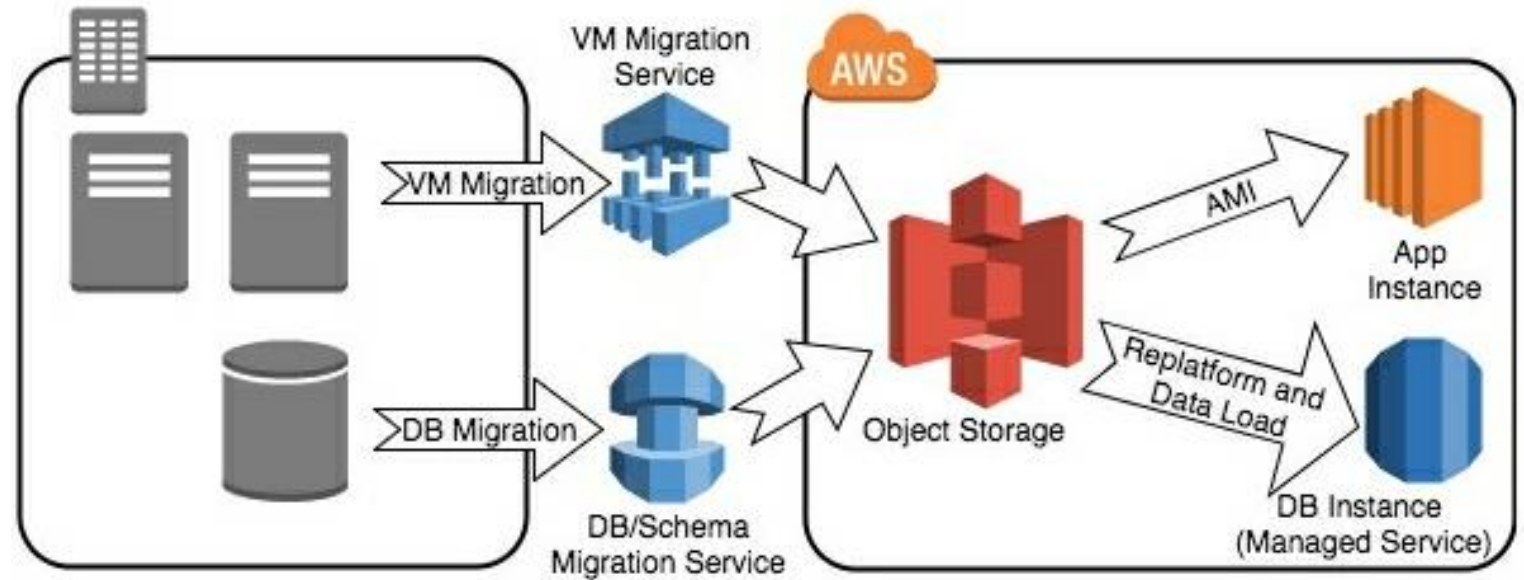


Changes to the workload architecture to take advantage of the cloud native vendor services (e.g., moving the application exactly as is, but changing the database from self-managed instances to a vendor-managed database platform)



While this change seems minor, it introduces a change to the workload architecture and must be managed as a mini-refactor with testing and documentation

Replatform



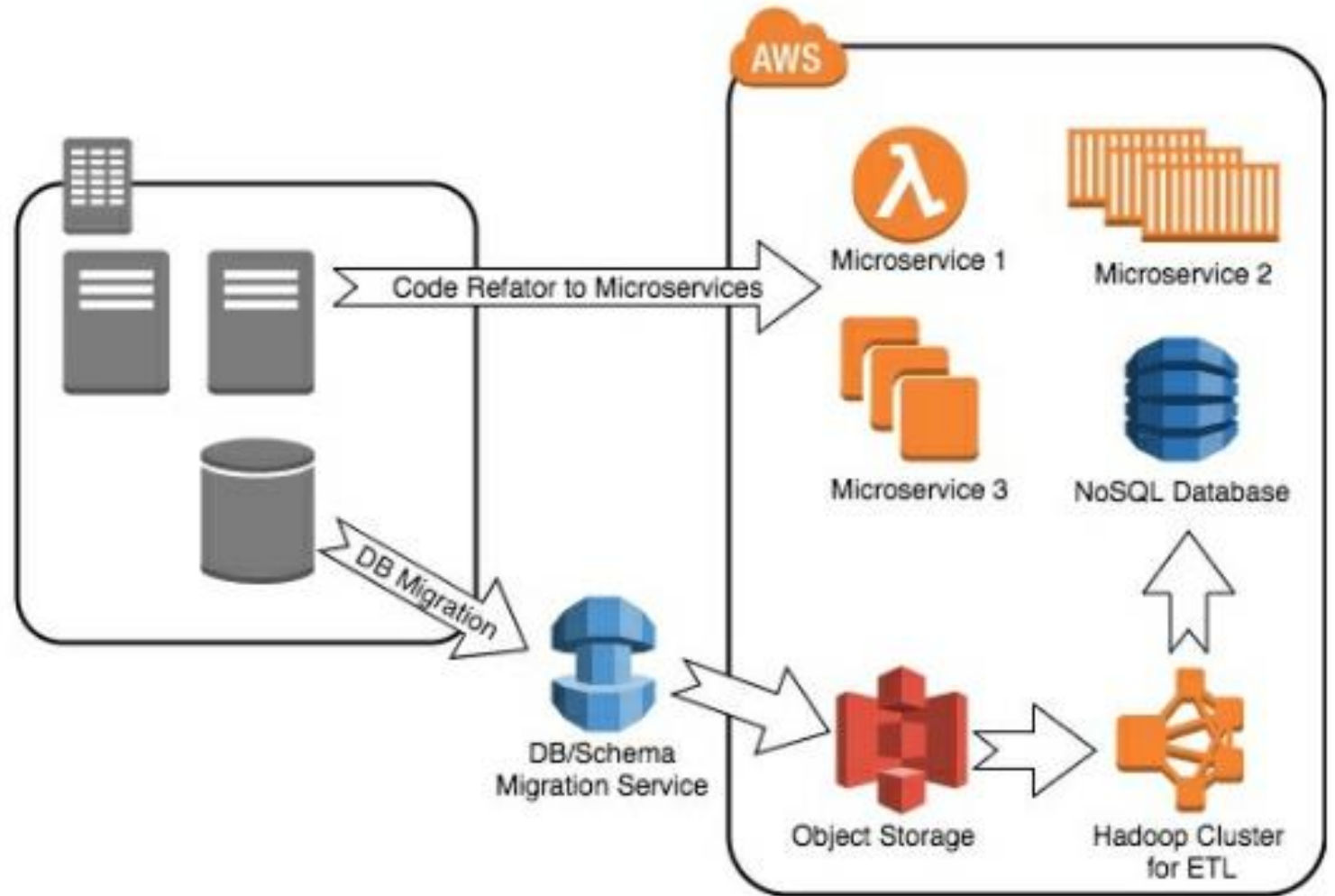
Refactor

The process of redesigning the workload to be cloud native

Rewrite of the application to align with the CNMM

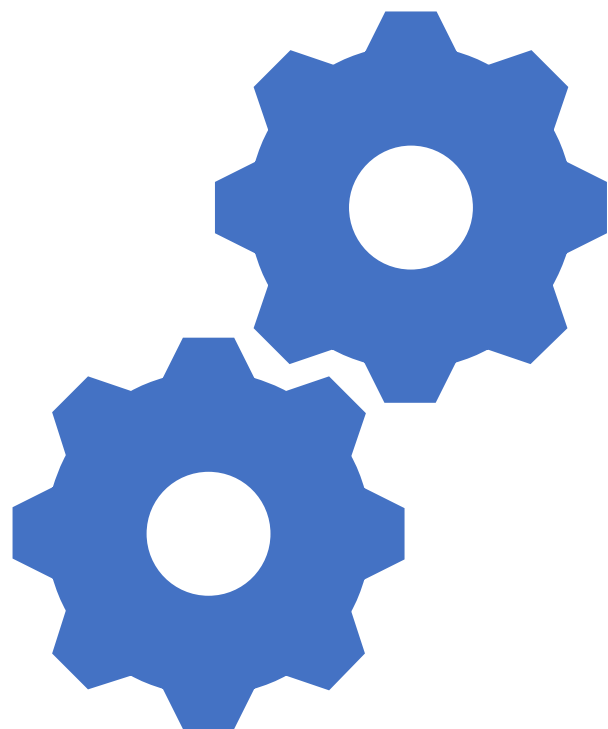
The application is being redesigned to decouple the service components and, most importantly, the business logic stays the same (or is enhanced) during this process

Refactor





Cloud Native Application Design



System design
patterns

Primitives

Base level of the overall solution: functions, tasks, microservices

A primitive is supposed to be the most basic unit that is capable of performing an action

Primitives are often so specialized that, by themselves, they are not capable of having a direct impact on the subsystem they belong to

They perform one task quickly and efficiently

Subsystems



Logical groupings of primitives that form a discrete business function



Do not have to be a single component (dependent on design architecture)



Need to have a logical flow that results in a business function being achieved

Systems

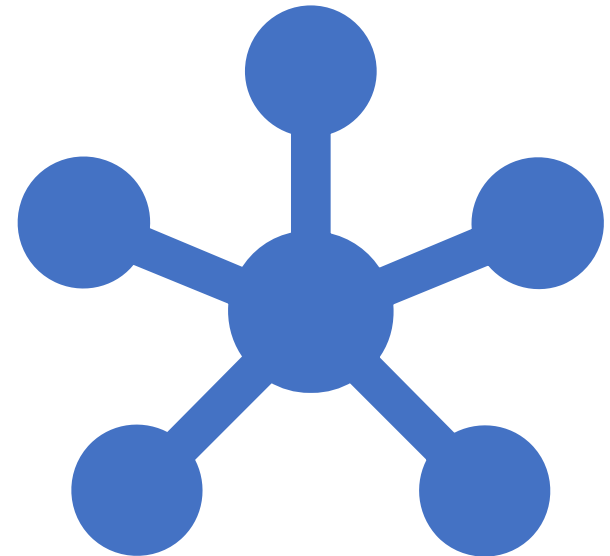
Are the top level of the solution

Are often made up of many subsystems that intercommunicate with each other

Complete an entire process from beginning to end

Main Design Patterns

- **Monolithic**
- **Client-server**
- **Services**
- All use systems, subsystems and primitives
- Key differentiator of the implementation of each => robust and cloud native



Monolithic

Characteristics



All aspects of the system are self-contained



Independent of other systems or processes



Worked great with mainframe monoliths

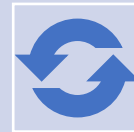


Networking connectivity did not exist



New business requirement, directly into existing system

Lesson Learned



Reusing components and tasks is a great way to save time, money, and reduce code change bugs



The concept of decoupling components is a key driving factor into the evolution of system designs

Client-Server



Monolith still exists for backend and database



The frontend application (client) is used to provide a user experience and transmit data to the server



Client tightly coupled with the backend



Allow for many clients to connect to the server



Skill sets started to specialize (client vs. server)

Services



Coupling of components causes slow and risky deployments for mission-critical applications



Separate components to reduce blast radius, increase speed of deployment and reduce risk

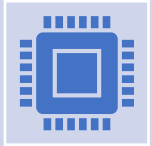


Services design pattern – business functions are broken down into specific objectives to complete in a self-contained manner that doesn't require knowledge of or dependencies **on the state of another service**



Key factors: reduction in costs of compute and storage, and increase in network sophistication and security practices for intercommunication

Cloud implications on Services



Cloud vendors can offer managed services to help remove the undifferentiated heavy lifting of some piece of a system



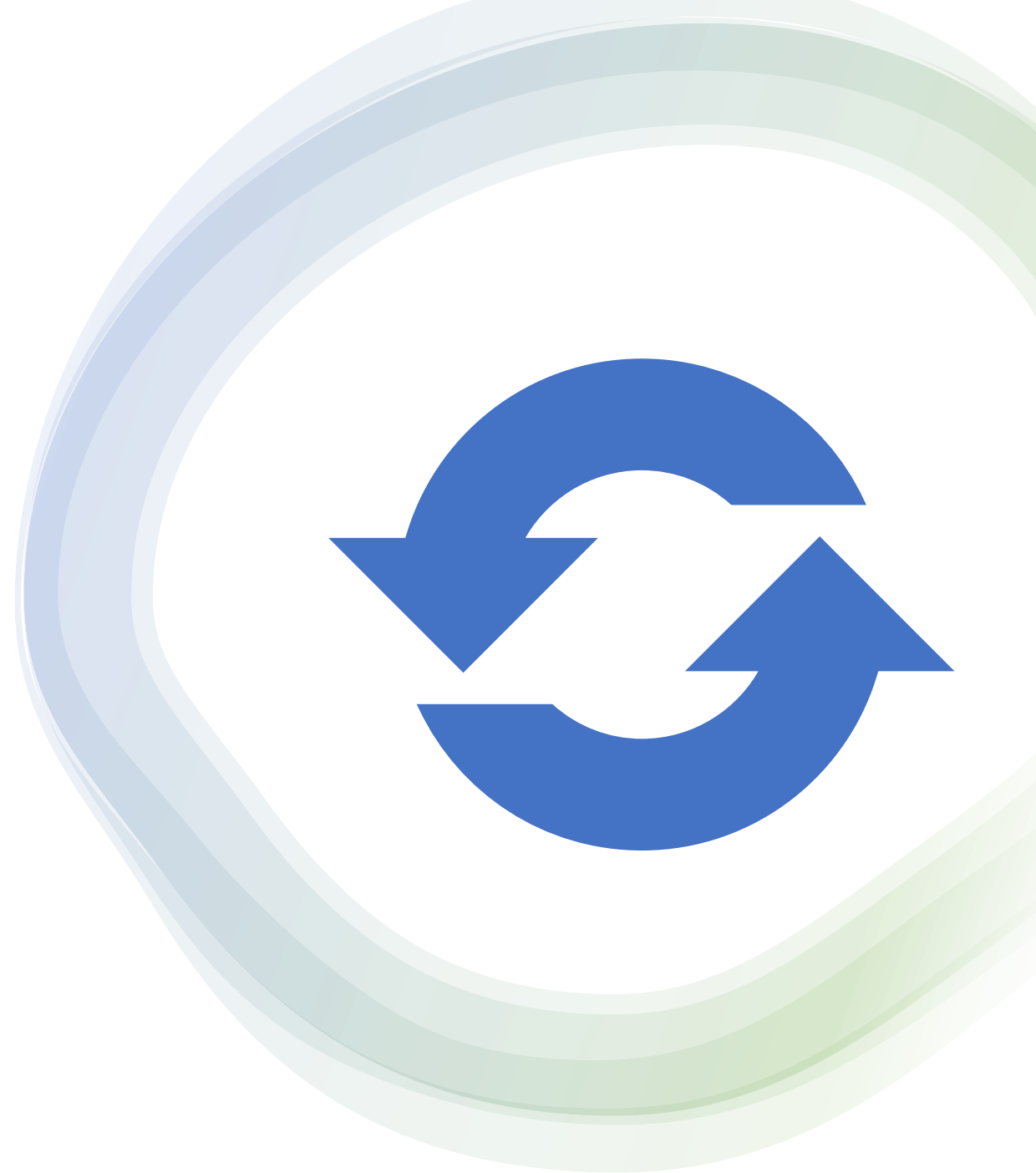
Examples: queues or storage services



Mature cloud native architectures are specifically designed to take advantage of these vendor services in addition to custom functions => reduce complexity

Evolution of service-based architecture

- SOAs -> autonomous services -> microservices
- Each pattern breaks up the functionality even more to decouple their interactions as much as possible



Service-oriented Architectures (SOA)

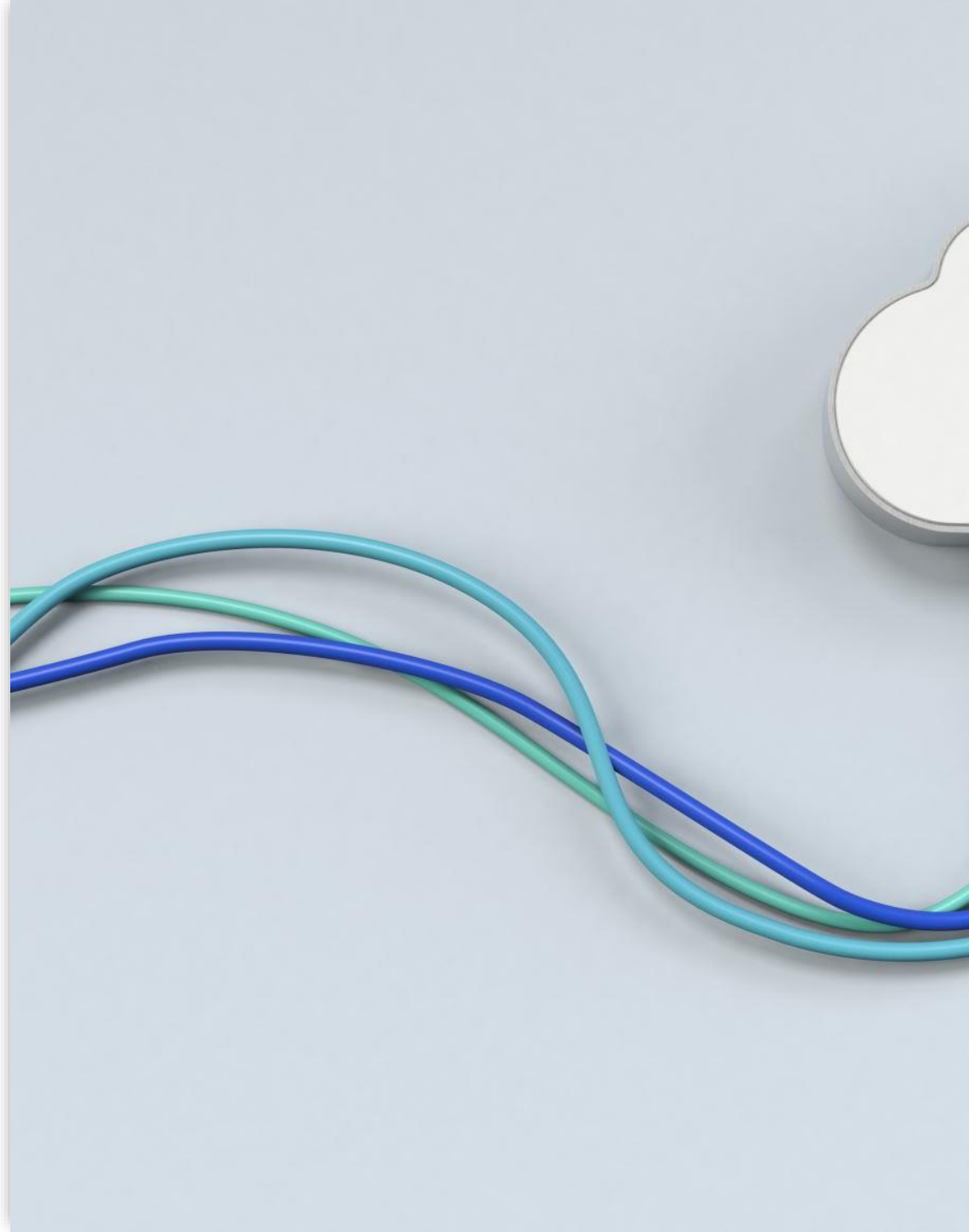
First major evolution of breaking the monolithic application

Still a large application split into subsystems that interact with each other just as they would in a monolithic system

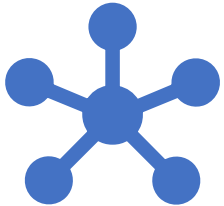
Use intermediation layer (Enterprise Service Bus - ESB)

- Message translation
- Routing
- Other forms of mediation

Individual services contain full sets of business logic => partially self-contained



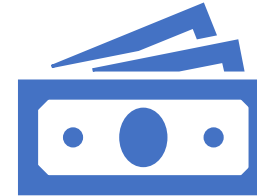
SOA benefits



Ability for systems to be
distributed



Serve business needs
faster



Take advantage of cheaper
compute resources

SOA drawbacks

ESB – single point of failure

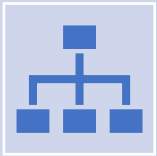
Message flow becomes very complicated

Any disruption to the ESB could cause significant slowdowns or queueing of messages

Testing the full composite application becomes very complicated and not isolated, slowing down business velocity

Changes in services propagate in ESB

Microservices



Breakdown of business functionality into smaller components



Key differences from SOA: the scope of the service and communication methods

Microservices: scope of a component

Breakdown subsystems into components closer to the primitives that perform a subset of the main service

A standalone component could not complete a full business task

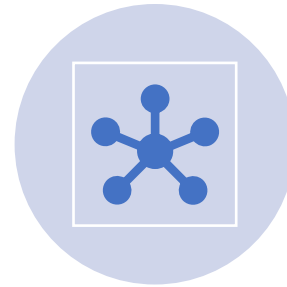
Allow for services to scale independently

Execution triggered based on an event => reduce time the resources are used

Microservices: communication method



APIs that can be called directly from any consumer service (no ESB)



APIs enable services to be developed in any language



Very little coordination between consumer services



When contracts are modified dependent services can be identified via dynamic API endpoint identification and navigation

Why Service Matters

Allow for
applications to be
designed into ever
smaller components

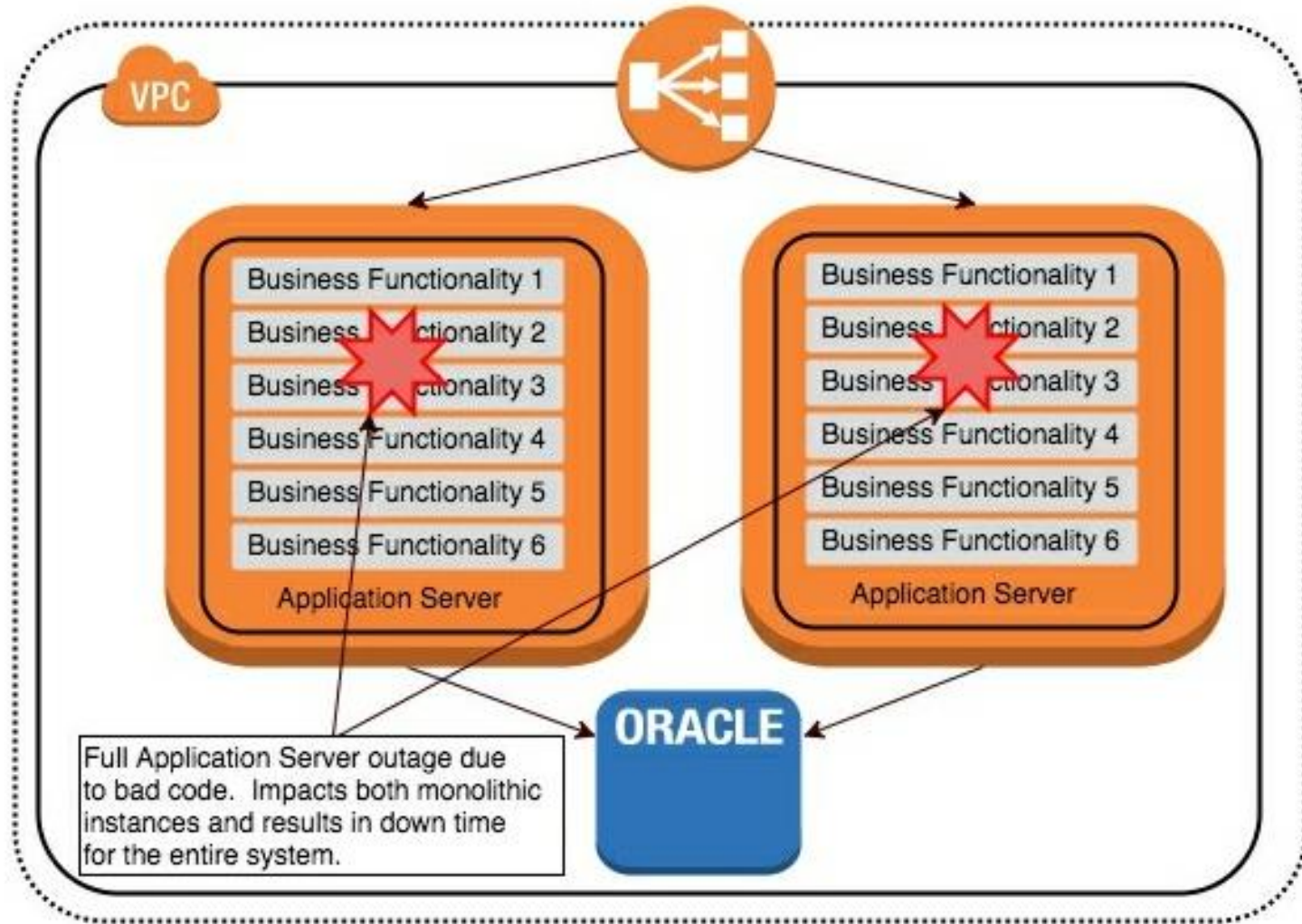


faster deployment velocity
business ideas come to market quicker

Smaller blast radius

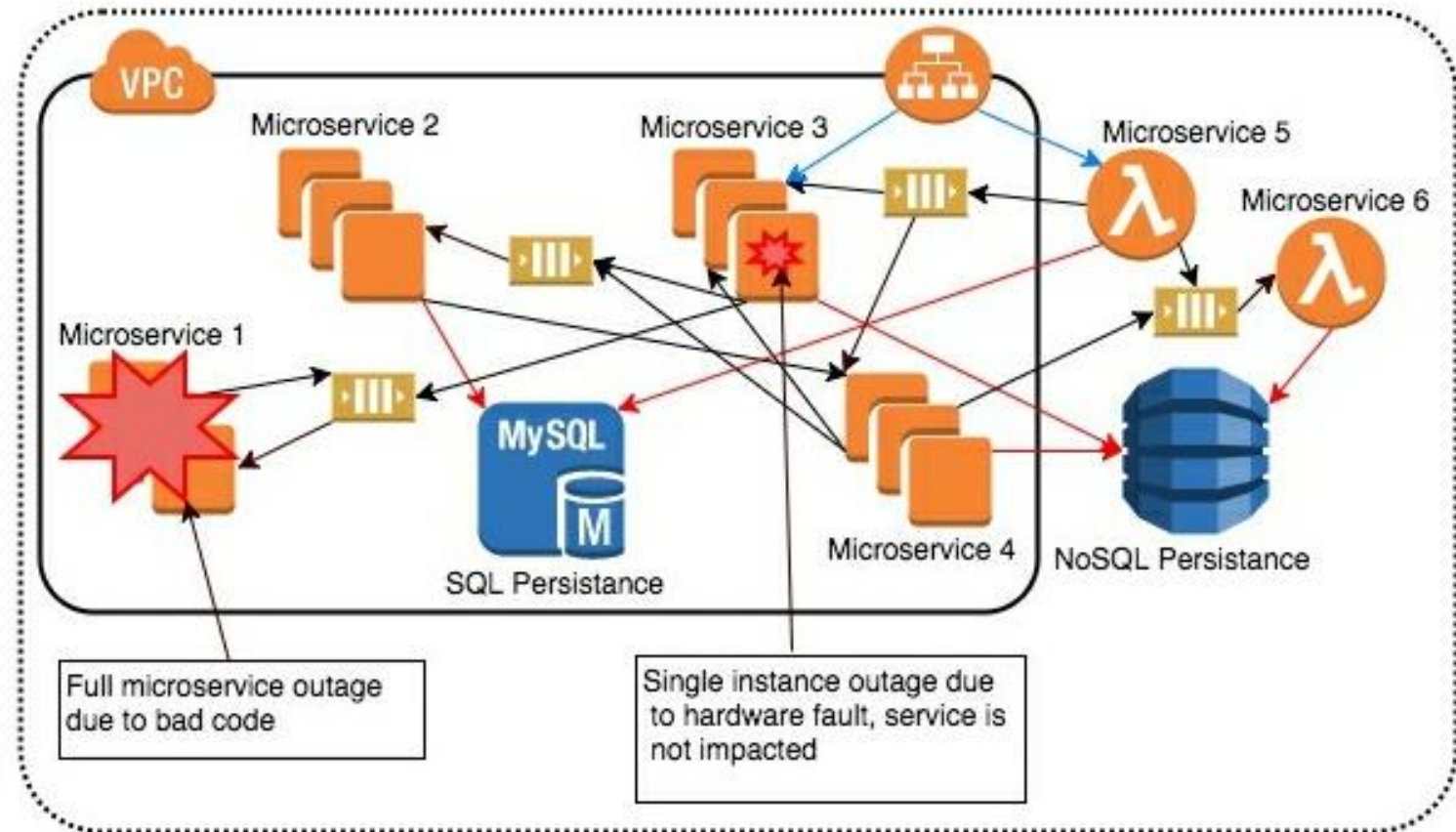
significant improvement on the risk profile of code deployments and
time needed to roll back

A monolithic system
deployed for
high
availability



Laszewski, Tom; Arora, Kamal; Farr, Erik; Zonooz, Piyum. Cloud Native Architectures: Design high-availability and cost-effective applications for the cloud. Packt Publishing.

Same
application
refactored as
microservices



Laszewski, Tom; Arora, Kamal; Farr, Erik; Zonooz, Piyum. Cloud Native Architectures: Design high-availability and cost-effective applications for the cloud. Packt Publishing.



Thank you